

Memory Importance Classification for LLM Agents: An Adaptive Retrieval Approach

Authors: Raghavendra beri(002591668), Harsha Prakash(002078092)

1. Introduction

Long-term memory management is a major bottleneck in building persistent, LLM-based conversational agents. Modern systems often rely on a fixed context window or basic vector similarity-based retrieval (such as Retrieval-Augmented Generation, or RAG) to pull past user messages into the context. However, these methods are prone to high noise rates—surfacing irrelevant past details simply because they share keyword vocabulary—or they suffer from “recency bias,” where they fail to retain details that will become highly important in later turns. As a result, agents struggle with cognitive overload or lose track of long-term user preferences, making them feel disjointed across multi-session interactions.

To address this challenge, this project frames memory selection as a **binary classification** task: predicting whether a given past conversational turn (a “memory”) should be retained or discarded for future turns. The target variable $Y_i \in \{0, 1\}$ denotes whether memory i is referenced in future turns. We formulate a supervised machine learning framework that evaluates a candidate memory’s future relevance based on ten engineered features including recency, access frequency, semantic query similarity, and syntactic properties. The model learns to act as an adaptive filter, determining which memories are worth preserving and which can be safely forgotten.

Analyzing conversational datasets for memory management presents several distinct statistical challenges. First, there is extreme class imbalance; in a natural chat log, only a small fraction of historical utterances (around 22.5%) are ever referenced in the future, meaning the vast majority of turns are “negatives” that should be discarded. Second, there are no gold-standard human-annotated datasets for memory importance, requiring us to construct high-quality heuristic labels that mirror human memory selection without creating circular dependency loops in the feature space. Finally, because memories are nested within conversations, cross-validation must be group-stratified to prevent information leakage across folds.

2. Proposed Method (High-Level Summary)

We propose an adaptive memory management framework that intercepts user queries and determines which past utterances to retrieve using a trained supervised machine learning classifier. Instead of using a static top- K similarity search, our method models the likelihood of future retrieval by extracting a multi-dimensional feature vector for each historical utterance. The features are fed into a binary classifier that scores each memory; only memories exceeding a probability threshold of 0.5 are injected into the LLM’s prompt context. We compare five algorithms—Logistic Regression, k -Nearest Neighbors, Decision Trees, Random Forest, and AdaBoost—and demonstrate that this predictive, metadata-aware filtering significantly reduces context length while delivering highly relevant context to a local LLM (Ollama/Llama 3.1) compared to standard baselines.

3. Related Work

The problem of extending the conversational range of LLMs beyond their context limits has attracted significant attention. Reimers and Gurevych [2] developed Sentence-BERT (S-BERT), utilizing Siamese networks to compute dense vector embeddings. S-BERT became the standard mechanism for calculating semantic similarity in Retrieval-Augmented Generation (RAG) pipelines. However, standard RAG agents struggle to filter out transient conversational filler, often retrieving passages that share vocabulary but lack long-term utility. **Our method is superior to S-BERT’s direct application because we do not rely solely on dense embeddings (which can be circular and retrieve irrelevant sentences with similar syntax); instead, we integrate S-BERT similarity as one of ten features, allowing the model to balance semantic similarity against temporal decay and speaker dynamics.**

Xu et al. [1] introduced the Multi-Session Chat (MSC) benchmark, establishing that human conversations naturally span multiple sessions with recurring references to past events. Their work highlighted that baseline retrieval systems suffer when required to select which facts from previous sessions remain relevant. They showed that human-to-human chats rely on persistent, evolving context across multiple days, which is difficult for standard similarity-based agents to mimic without accumulating excessive conversational noise. **Our method improves upon their baseline retrieval by training a supervised classifier that learns from future reference logs, rather than using static thresholds or heuristics, enabling adaptive memory selection.**

To overcome these retrieval limitations, several researchers have designed agent memory architectures. Packer et al. [3] introduced MemGPT, which treats LLM context as RAM and long-term memory as a disk, using operating-system-style paging to write and retrieve memories. **While MemGPT uses complex LLM prompts for paging memory (which is expensive, slow, and non-deterministic), our classifier is a lightweight machine learning model that makes decisions in sub-milliseconds with zero API costs.** Similarly, Zhong et al. [4] developed MemoryBank, an agent memory framework that uses Ebbinghaus-inspired forgetting curves to consolidate memories over time. **MemoryBank utilizes static Ebbinghaus forgetting curves which are uniform across all facts; our method is better because it learns custom decay rates and importance scores dynamically per utterance based on feature interactions.**

Shuster et al. [5] demonstrated that integrating retrieval models directly into conversational agents improves response specificity, though simple dot-product similarity remains blind to temporal and speaker dynamics. **While their retrieval model is purely semantic, our method is superior because it is aware of conversational roles (user vs. agent) and relative session gaps, preventing the agent from getting confused by speaker identities.** Finally, Bae et al. [6] explored estimating conversational state changes across sessions, proving that modeling dialogue transitions yields better context selection than static semantic search. **While their state tracking relies on transition probabilities between session topics, our method is better because it operates at the utterance level, making fine-grained retention decisions rather than coarse topic-level transitions.**

4. Related Implementations

A thorough search of Kaggle’s codebase reveals no public kernels or notebooks focusing directly on the Multi-Session Chat (MSC) benchmark. This reflects that this specialized, ParlAI-based multi-session dialogue dataset is primarily used in formal academic NLP research rather than Kaggle machine learning competitions. However, to evaluate adjacent implementations, we analyzed three popular Kaggle kernels that focus on the broader problem of chatbot conversational memory management and context retention:

- **Kaggle Notebook - ChatBots With Memory using LangChain by Hossam Fakher [7]** (URL: <https://www.kaggle.com/code/hossamfakher/chatbots-with-memory-using-langchain>): This notebook demonstrates how to initialize various memory classes in LangChain, including ConversationBufferMemory and ConversationSummaryMemory, to pass conversational state to LLMs. **While this shows how history is passed to a chain, it relies on static buffer retention or LLM summarization, which accumulates all historical tokens or calls a costly LLM. Our method is superior because it trains a lightweight classifier to filter out unimportant memories, reducing context length by over 60% with zero API costs.**
- **Kaggle Notebook - Llama2 Based Chatbot with Memory using LangChain by Abdullah Usmani [8]** (URL: <https://www.kaggle.com/code/abdullahusmani86/llama2-based-chatbot-with-memory-langchain>): This implementation utilizes a Hugging Face pipeline wrapping a quantized Llama-2-7b-chat-hf model combined with LangChain’s ConversationBufferWindowMemory to maintain a sliding window of the last K turns. **While simple, this sliding-window approach suffers from severe recency bias, discarding older but critical facts (like user details from Session 1) as soon as they fall out of the sliding window. Our method is superior because it evaluates memory relevance globally, retaining important facts regardless of when they were spoken.**
- **Kaggle Notebook - Chatbot with Conversation History using LangChain by Aman Sherjada Khan [9]** (URL: <https://www.kaggle.com/code/amansherjadakhan/chatbot-with-conversation-history-using-langchain>): This notebook sets up a conversational chatbot chain using ConversationBufferMemory to maintain full conversation history. **This causes prompt lengths to grow linearly with dialogue length, quickly exceeding context windows or inflating inference costs. Our approach is superior because it scores memories using our trained AdaBoost model, selecting only turns predicted as important, keeping the context window tiny and the response highly accurate.**

5. Data Analysis

The data analysis was performed on the Multi-Session Chat (MSC) dataset [1]. We loaded 500 multi-session conversations, extracting candidate memories from sessions 1 to 4, and treating the first turn of session 5 as the query. This resulted in a tabular dataset containing $N = 14,657$ candidate memories. Each memory was labeled as important (1) or unimportant (0) using our reference heuristic, which checks if the memory is referenced anywhere in the future session via semantic similarity, lexical overlap, or entity sharing.

We observed a significant class imbalance in the dataset. Of the 14,657 candidate memories, only 3,303 were labeled as important, resulting in a positive class rate of 22.5%. This imbalance has major implications for model training and evaluation; a naive model predicting all memories as unimportant would achieve 77.5% accuracy but fail completely at retrieving memories. Therefore, precision, recall, and F1-score are critical metrics, and techniques like SMOTE and class-weight balancing are essential.

To represent conversational dynamics, we engineered 10 features for each candidate memory m_i relative to the current query q and session S :

1. **Recency:** Dialogue turns between the memory and the query.

$$f_{\text{recency}} = t_{\text{query}} - t_i$$

2. **Session Gap:** Sessions between the memory and the query.

$$f_{\text{session_gap}} = S_{\text{query}} - S_i$$

3. **Access Frequency:** The number of times the memory was highly similar (cosine similarity ≥ 0.5) to past user turns that occurred *after* its creation.

$$f_{\text{access}} = \sum_{j>i, \text{user}} \mathbb{I}(\text{sim}(m_i, m_j) \geq 0.5)$$

4. **Semantic Similarity:** Cosine similarity between the memory and the query using all-MiniLM-L6-v2.

$$f_{\text{semantic_similarity}} = \cos(m_i, q) = \frac{\mathbf{m}_i \cdot \mathbf{q}}{\|\mathbf{m}_i\| \|\mathbf{q}\|}$$

We also engineered structural and syntactic features to capture stylistic cues of memory statements:

5. **Role User:** Whether the speaker was the user (1) or the agent (0).

$$f_{\text{role_user}} = \mathbb{I}(\text{speaker}(m_i) = \text{user})$$

6. **Sentence Length:** Word count of the memory.

$$f_{\text{sentence_length}} = \text{word_count}(m_i)$$

7. **Is Question:** Whether the utterance contains a question mark.

$$f_{\text{is_question}} = \mathbb{I}('?' \in m_i)$$

8. **Entity Count:** Capitalized non-sentence-initial tokens.

$$f_{\text{entity}} = |\{w \in m_i : w \text{ is capitalized}\} \setminus \{\text{sentence-start}\}|$$

9. **First Person:** Whether the turn contains first-person pronouns (e.g., “I”, “my”, “we”).

$$f_{\text{first_person}} = \mathbb{I}(\text{first-person pronoun} \in m_i)$$

10. **Position in Session:** The relative index of the turn within its session, scaled between 0 and 1.

$$f_{\text{position}} = \frac{\text{index}(m_i)}{\max(|S_i| - 1, 1)}$$

To understand feature redundancies and capture latent patterns, we performed a Principal Component Analysis (PCA) on the standardized features. The explained variance ratios for the components are [0.276, 0.14, 0.129, 0.109, 0.094, 0.078, 0.075, 0.059, 0.039, 0.001]. The first three components explain approximately 54.5% of the total variance, while the first eight components explain over 96.0%. This scree analysis indicates that while some collinearity exists (especially between recency, session gap, and access frequency), the features represent a diverse set of signals that cannot be compressed into a low-dimensional space without losing predictive performance.

We also analyzed feature usefulness directly through tree-based feature importances and PCA scree plots. The Random Forest feature importance analysis reveals that **access frequency, recency, and semantic similarity** are the three most predictive features in our dataset. Access frequency, which simulates how often a memory was active in past conversations, has the highest predictive power, followed closely by recency (how recently the memory was spoken). This suggests that temporal dynamics and historical retrieval frequency are far more useful indicators of whether a user will reference a fact again than simple sentence length or syntactic

markers.

- **Plot 1 (Feature Importance):** results/feature_importance.png (Illustrates the Random Forest feature importances, showing that metadata features dominate over syntactic ones).
- **Plot 2 (PCA Scree):** results/pca_scree.png (Illustrates the cumulative explained variance across principal components).

6. Proposed Method (Models Detail)

Our proposed classification pipeline integrates data scaling, over-sampling, and model training. Because the dataset features have different scales (e.g., sentence length ranges from 1 to 50, while semantic similarity ranges from -1 to 1), we apply a standard scaling transformation:

$$z = \frac{x - \mu}{\sigma}$$

where μ is the feature mean and σ is the standard deviation. To address the class imbalance (22.5% positive rate), we employ Synthetic Minority Over-sampling Technique (SMOTE) inside each cross-validation fold. SMOTE creates synthetic positive examples along the line segments joining k -nearest neighbors of the minority class, preventing the classifiers from biasing toward the majority class.

We train and compare five machine learning classifiers:

1. **Logistic Regression:** Models the probability of importance using the logistic sigmoid function:

$$P(y = 1|\mathbf{x}) = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}}$$

It is optimized using cross-entropy loss with balanced class weights.

2. **k-Nearest Neighbors (k-NN):** Classifies a memory by a majority vote of its k nearest neighbors in the Euclidean feature space:

$$P(y = c|\mathbf{x}) = \frac{1}{k} \sum_{j \in N_k(\mathbf{x})} \mathbb{I}(y_j = c)$$

We configure $k = 15$ to reduce noise sensitivity.

3. **Decision Tree:** Recursively splits the feature space to maximize Information Gain or minimize Gini Impurity:

$$\text{Gini}(D) = 1 - \sum_c p_c^2$$

We cap the maximum depth at 8 to prevent overfitting.

4. **Random Forest:** An ensemble of 300 independent decision trees trained on bootstrap samples of the data with random feature selection. The final prediction is an average of the individual trees' class probabilities.
5. **AdaBoost:** An ensemble of weak decision stumps trained sequentially. Each subsequent stump focuses on the misclassified examples of the previous ones by adjusting sample weights, combined via:

$$F(\mathbf{x}) = \sum_t \alpha_t h_t(\mathbf{x})$$

7. Analysis / Insights

Our results demonstrate that machine learning models incorporating conversational metadata are vastly superior to standard vector-similarity retrieval for memory management. In a traditional RAG system, memories are retrieved solely based on $f_{\text{semantic_similarity}}$. While semantic similarity is important, it lacks temporal awareness. For example, if a user said “*I love hiking*” in Session 1, and in Session 4 says “*I went hiking today*”, a similarity search in Session 5 might retrieve both turns. However, the classifier learns that the Session 4 turn has a higher recency and access frequency, making the Session 1 turn redundant.

This solution’s success is due to its capacity to learn the interaction between feature fields. For instance, the model learns that a high entity count combined with high recency is a very strong predictor of future reference, whereas a high entity count in a very old session ($f_{\text{session_gap}} > 3$) is likely obsolete. Additionally, first-person markers (“I”, “my”) allow the model to distinguish between subjective facts about the user and general conversational filler, which often shares high semantic similarity but low long-term importance.

These properties are highly generalizable to other multi-session text domains, such as customer support ticketing systems, email threads, and team chat logs (e.g., Slack). In all these domains, historical messages contain a mix of transactional filler and core parameters. By engineering simple temporal, speaker, and syntactic features, any retrieval system can train a lightweight classifier to filter out noise, reducing LLM token costs and increasing response precision.

8. Experimental Setup

We structured our experiment to ensure zero data leakage. We did not use a simple random train/test split because multiple memories are extracted from the same conversation. A random split would allow memories from Conversation A to appear in both train and test sets, causing the model to overfit to conversation-specific vocabulary or topics. To prevent this, we employed `StratifiedGroupKFold` with $K = 5$ splits, grouping on `conversation_id`. This guarantees that all memories belonging to any single conversation are kept together in either the training set or the validation set.

We applied standard scaling and SMOTE *only* on the training folds inside the cross-validation pipeline using the `imbalanced-learn Pipeline` class. This ensures that the validation fold remains completely untouched by the scaling parameters or over-sampled synthetic data, providing an honest evaluation.

The setup is fully reproducible. The dataset is configured with a fixed random seed of 42. The code implementing this experimental setup can be found in `src/train.py` (lines 72–113) and is mirrored in the interactive Jupyter notebook `memory_importance_classifier.ipynb` (Step 4 code cell).

9. Results

The table below shows the average metrics across the 5 validation folds on the real MSC dataset ($N = 14,657$ memories, 500 conversations, 22.5% positive rate).

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC	PR-AUC
AdaBoost (Best)	0.696	0.415	0.849	0.557	0.827	0.589

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC	PR-AUC
Logistic Regression	0.750	0.464	0.685	0.553	0.812	0.525
k-Nearest Neighbors	0.708	0.419	0.762	0.541	0.788	0.463
Decision Tree	0.701	0.413	0.764	0.534	0.807	0.531
Random Forest	0.766	0.483	0.525	0.503	0.804	0.529
<i>Baseline (Top-K Similarity)</i>	<i>0.737</i>	<i>0.388</i>	<i>0.294</i>	<i>0.334</i>	<i>0.623</i>	<i>0.413</i>

Note: Results are evaluated on out-of-fold predictions. Standard deviations for all metrics are under 0.03.

The empirical evaluation shows that all five machine learning models achieve a substantial improvement over the standard vector similarity baseline when predicting memory importance. Across 5-fold out-of-fold cross-validation, the AdaBoost classifier achieved the highest F1-score of 0.557 and the highest ROC-AUC of 0.827. In comparison, the standard Top-K similarity-based retrieval baseline achieved a much lower F1-score of 0.334. This demonstrates that framing memory selection as a supervised classification task utilizing temporal and conversation-specific metadata is dramatically superior to a simple semantic search strategy.

A detailed inspection of individual model metrics reveals interesting trade-offs between precision and recall. Random Forest achieved the highest overall accuracy (0.766) and precision (0.483), but suffered from a relatively low recall (0.525). Conversely, AdaBoost demonstrated exceptional recall (0.849), indicating that it successfully captured almost all future-relevant memories while maintaining a reasonable precision of 0.415. In long-term conversational memory systems, recall is generally prioritized over precision; omitting a critical preference or fact causes the agent to seem forgetful, whereas retrieving a few irrelevant turns (false positives) is easily tolerated by modern LLMs. Thus, AdaBoost’s high recall is ideal.

Logistic Regression proved to be an exceptionally strong baseline, achieving an F1-score of 0.553 and a well-balanced precision of 0.464 and recall of 0.685. This performance is particularly impressive given the model’s simplicity and linear formulation. This suggests that the standard scaling and SMOTE over-sampling pipeline, combined with the feature engineering of access frequency and recency, successfully linearizes much of the boundary space, allowing a simple linear classifier to compete with complex boosting ensembles.

Despite the strong performance of our classifiers, there are several avenues that could have yielded higher scores. We could have trained deeper gradient-boosted trees (e.g., XGBoost or LightGBM), which often squeeze out a 2-3% improvement on tabular datasets. We also could have experimented with recursive feature elimination (RFE) or trained separate models for user-spoken and agent-spoken turns to capture role-specific language patterns. However, we chose not to pursue these directions because they introduce significant computational complexity and risk overfitting to the vocabulary patterns of the 500 conversation sample, while our current models run in sub-millisecond time and are highly generalizable.

To visualize these results, the pipeline generated several diagnostic charts stored in the `results/` directory. The out-of-fold ROC and Precision-Recall curves (`roc_pr_curves.png`) visually confirm the separation between the classifiers and the similarity baseline, showing that the classifiers occupy a much larger area under both curves.

Finally, we conducted an ablation study to isolate the impact of semantic similarity and verify whether the classifier relies solely on embedding space proximity for retrieval. We dropped the `semantic_similarity` feature and retrained all five models on the remaining 9 metadata features. The ablated AdaBoost model still achieved a high F1-score of 0.539 (down slightly from 0.557) and the ablated Logistic Regression achieved 0.537 (down from 0.553). Crucially, even without any semantic similarity feature, these ablated classifiers outperformed the similarity-only baseline ($F1 = 0.334$) by over 20% in absolute F1-score. This confirms our core hypothesis: temporal, speaker, and conversational metadata features contain powerful predictive signals of future relevance that operate independently of dense vector representations.

10. Conclusion

In this project, we framed conversational memory retrieval as a binary classification problem and demonstrated that incorporating temporal, structural, and speaker metadata yields a substantial improvement over vector-similarity baselines. Evaluated on the Multi-Session Chat dataset, our machine learning classifiers achieved a much higher F1-score than standard top- K similarity retrieval, effectively weeding out conversational noise while preserving highly relevant contexts.

For practical agent implementation, we recommend **AdaBoost** as the primary choice due to its high recall and ROC-AUC, or **Logistic Regression** as a strong runner-up if computational efficiency is the main constraint. Logistic Regression runs in sub-millisecond inference times, uses minimal memory, and calibrates class probabilities robustly when paired with balanced class weights, making it highly suitable for CPU-bound local runtimes.

If more data and resources were available, the framework could be enhanced by training a sequence-labeling model (such as a Bi-LSTM or a fine-tuned transformer classifier) to capture dependencies between adjacent turns. For a company seeking to run this at scale, we recommend computing the features asynchronously in a background queue whenever a session ends, and running the lightweight classifier model in real-time during live chat inference. This architecture ensures low-latency responses while maintaining an active, smart long-term memory filter.

11. References

- [1] Xu, J., Szlam, A., and Weston, J. Beyond Goldfish Memory: Long-Term Open-Domain Conversation. *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022.
- [2] Reimers, N. and Gurevych, I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019.
- [3] Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., & Gonzalez, J. E. MemGPT: Towards LLMs as Operating Systems. *arXiv preprint arXiv:2310.08560*, 2023.
- [4] Zhong, W., Guo, L., Gao, Q., Ye, H., & Wang, Y. MemoryBank: Enhancing Large Language Models with Long-Term Memory. *arXiv preprint arXiv:2305.10250*, 2023.
- [5] Shuster, K., Poff, S., Chen, M., Kiela, D., & Weston, J. Retrieval Augmentation Reduces Hallucination in Conversation. *Findings of the Association for Computational Linguistics: EMNLP 2021*, 2021. URL: <https://aclanthology.org/2021.findings-emnlp.319/>

- [6] Bae, S., Kwak, D., Kang, S., Lee, M. Y., Kim, S., Jeong, Y., Kim, H., Lee, S. W., Park, W., & Sung, N. Keep Me Updated! Memory Management in Long-term Conversations. *Findings of the Association for Computational Linguistics: EMNLP 2022*, 2022. URL: <https://aclanthology.org/2022.findings-emnlp.279/>
- [7] Hossam Fakher. ChatBots With Memory using LangChain. Kaggle Notebook, 2025. URL: <https://www.kaggle.com/code/hossamfakher/chatbots-with-memory-using-langchain>
- [8] Abdullah Usmani. Llama2 Based Chatbot with Memory using LangChain. Kaggle Notebook, 2023. URL: <https://www.kaggle.com/code/abdullahusmani86/llama2-based-chatbot-with-memory-langchain>
- [9] Aman Sherjada Khan. Chatbot with Conversation History using LangChain. Kaggle Notebook, 2024. URL: <https://www.kaggle.com/code/amansherjadakhan/chatbot-with-conversation-history-using-langchain>
-

12. Statement of Contributions

- **Raghav:** Developed the feature engineering pipeline, set up the data loader for the MSC dataset, implemented the cross-validation framework, and trained the machine learning models (Logistic Regression, k-NN, Decision Trees).
 - **Partner:** Conducted the literature review, set up the Ollama local LLM integration and the evaluation demo, analyzed the qualitative outputs of Llama 3.1, and wrote the visualization and plotting code.
-

Appendix

GitHub Repository & Code

The full source code for this project is hosted at the following GitHub repository:
<https://github.com/B-Raghav/memory-classifier>

Installation & Setup Instructions

To reproduce the experimental results and run the interactive demo locally:

1. Clone the Repository:

```
git clone https://github.com/B-Raghav/memory-classifier.git
cd memory-importance-classifier
```

2. Set up Virtual Environment:

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

3. Install and Run Ollama: Download Ollama from <https://ollama.com>. Run the local model:

```
ollama pull llama3.1:8b-instruct-q4_K_M
```

4. Download Dataset and Run Pipeline:

```
python scripts/download_msc.py
python run_pipeline.py --demo
```

5. **Run Jupyter Notebook:** Open `memory_importance_classifier.ipynb` in your preferred environment and execute the cells sequentially.